NVIDIA ASIC Verification Engineer

Question 1 (Python) Write a function that takes a list of regression results

```
'''
Input
'''
results = [
    {"test": "axi_smoke",  "seed": 1, "status": "PASS", "signature": ""},
    {"test": "axi_burst",  "seed": 3, "status": "FAIL", "signature": "timeout"},
    {"test": "fifo_stress","seed": 5, "status": "FAIL", "signature": "fifo_mismatch"},
]
'''
Output
'''
{
  "timeout":         [("axi_burst", 1)],
  "fifo_mismatch":   [("fifo_stress", 5)],
}
```

My answer:

```
'''
Since we have a dict (key/value pair), we can use methods in collections to create a dict
then check for keyword "FAIL" and append relevant fields into output dict.
'''
from collections import defaultdict

def group_failures(results):
    groups = defaultdict(list)
    for r in results:
        if r["status"] == "fail":
            groups[r["signature"]].append((r["test"], r["seed"]))
    return dict(groups)
```

Later on he asked about checking corner cases such as `signature` is `UNKNOWN` or `seed` is `null`. Did not have time and moved on to next question.

Question 2 (SystemVerilog/UVM) Given a FIFO interface

```
input logic clk;
input logic reset;
input logic wr_en;
...
input logic [7:0] rd_data
```

Write a scoreboard that check the FIFO rules

1. A write is accepted when `wr_en && !full`,
2. A read returns data when `rd_en && !empty`,
3. Data must come back in the same order it was written.

My Answer:

```
// At first I confirmed whether it's a UVM scoreboard or not, he said no and I was confused.
// My implementation finally came out to be something like this (incomplete):
```

```
typedef struct {
  // Signals in interface
  logic [7:0] wr_data;
  logic [7:0] rd_data;
  bit         wr_en;
  bit         rd_en;
  bit         full;
  bit         empty;
} interface_data;

class fifo_scb;
  logic [7:0] model[$];

  // Rule 1
  function bit check_write(interface_data id);
    if (id.wr_en && !id.full) begin
      model.push_back(id.wr_data);
      return 0;
    end
    else begin
      `uvm_error("SB", "FIFO FULL")
      return 1;
    end
  endfunction

  // Rule 2 & 3
  function bit check_read(interface_data id);
    if (id.rd_en && !id.empty) begin
      if (id.rd_data == model.pop_front()) begin
        return 0;
      end
      else begin
        `uvm_error("SB", "READ MISMATCH")
        return 1;
      end
    end
    else begin
      `uvm_error("SB", "Unexpected READ")
      return 1;
    end
  endfunction

  // Top-level check
  function bit overall(interface_data id);
    bit w, r;
    w = check_write(id);
    r = check_read(id);
    return (w || r);
  endfunction
endclass

// How to use
fifo_scb scb = new();
result = scb.overall(id);
```